

Cloud Computing Project – Review 1

Multiplayer Robot Arena: programmable robot combat as a cloud service

Complete project report · Phase 1 prototype

Project Title

Multiplayer Robot Arena (repository: `cloud-robot`).

A browser-based multiplayer robot arena in the style of a programming game. Every signed-in player owns one persistent, resource-limited SSH development box in the cloud, writes a robot brain in Lua 5.4 at `/workspace/main.lua`, and deploys it into a shared arena. A server-authoritative Go engine runs the match at a deterministic 10 Hz tick and streams versioned state snapshots to every connected browser, so players and public spectators watch the same fight in near real time. All cloud dependencies are AWS-compatible services (S3, DynamoDB, SQS) exercised locally through Floci, keeping the stack deployable later to a single VPS without code changes.

Team Details

Name	Roll number	Branch	Responsibility
Team member 1	21XXXXXXX	CSE	Simulation engine, bots, maps, items
Team member 2	21XXXXXXX	CSE	API server, duel queue, cloud adapters, auth
Team member 3	21XXXXXXX	CSE	Svelte 5 web client, spectating, docs site
Team member 4	21XXXXXXX	CSE	Box provisioning, supervisor, Lua SDK, examples

Review Details

Review	Review 1 (expanded demo)
Course	Cloud Computing
Date	DD Month 2026
Faculty guide	Guide name
Demo scope	Two SSH boxes scripted over SSH, two synchronized browsers, Floci-backed persistence, practice match against a server bot, logged-out spectating, post-match recap

Introduction

What the project is

Multiplayer Robot Arena turns a classic programming-game concept into a cloud application. Instead of running robot code on a laptop, each player gets a cloud-hosted development box and joins a shared simulation hosted by a Go API server. The robot is ordinary Lua 5.4 code; the arena is an ordinary browser page; the match state lives only on the server.

The player loop

1. **Provision** — sign in (WorkOS AuthKit) and receive one persistent Docker SSH box with a working demo robot already at `/workspace/main.lua`.

2. **Program** — edit the robot over SSH, or deploy curated persona scripts (aggressive, demolisher, evasive, patroller, sentinel, sniper) from the box console. The server syntax-checks Lua on deploy.
3. **Register** — enrollment snapshots the script to S3, stores only a SHA-256 hash of the robot token in DynamoDB, and starts the in-box agent. The token never reaches the browser.
4. **Fight** — the agent keeps one authenticated outbound WebSocket to the API and exchanges observations for intents every tick.
5. **Watch** — browsers receive the same 10 Hz snapshots; finished matches keep event replays, per-robot stats, and rating updates.

Why this is a cloud computing project

- **Managed-service patterns:** object storage for script snapshots, a document table for match and player state, and a queue for match jobs — all accessed through small Go adapters so Floci (local AWS emulation) can later be replaced by real AWS with configuration changes only.
- **Asynchronous job flow:** match creation enqueues a job; a worker consumes it, provisions containers, runs the simulation, and persists results.
- **Multi-tenant isolation:** one container per player with workspace quotas; Phase 1 treats boxes as trusted-reviewer hardware while a Firecracker microVM phase is planned for real isolation.
- **One-command operations:** mise tasks start the API, web client, and Floci stack locally; Docker Compose underpins the later VPS deployment.

Problem Statement

Programming-game platforms such as Robocode or Battlecode run all robot code on one machine. That shape teaches none of the service architecture of real cloud games and offers no shared live view for spectators. Concretely, the project must solve:

- **Untrusted-ish code execution:** player scripts run inside per-player boxes with CPU/memory/workspace quotas, and the robot token stays inside the box — never in the browser, never in the engine.
- **Server authority:** agents return **intent only** (move, turn, fire, dash, deploy, scan, message). The engine alone computes position, damage, score, and match state, so a lying agent gains nothing.
- **Synchronized spectating:** any number of anonymous browsers must observe the same authoritative state with near-real-time latency, without being able to influence it.
- **Persistence through cloud services:** script snapshots, box metadata, match jobs, results, and player statistics must survive stack restarts and live in AWS-compatible storage, not local files.
- **Determinism for grading:** every random system (maps, items, crits, bots) seeds from match configuration; replays are byte-stable so a match can be re-examined tick by tick.

Objectives

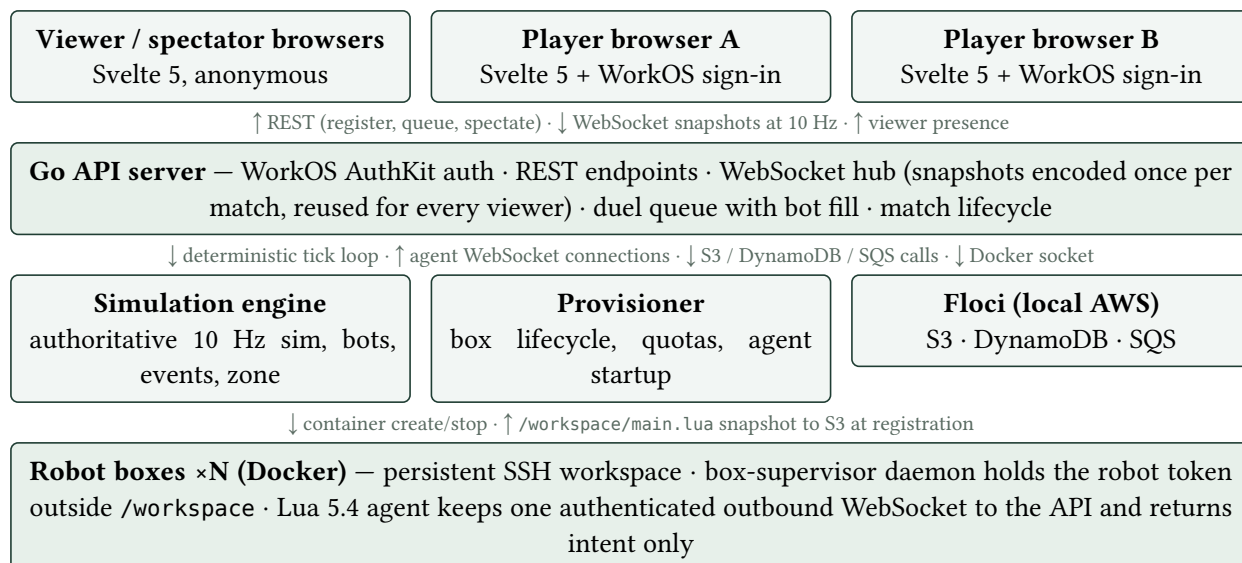
1. Deliver a complete demoable loop: provision box → edit Lua over SSH → register robot → synchronized live match in two browsers.
2. Keep the simulation server-authoritative and seed-deterministic at 10 Hz, with byte-stable event logs for replays.
3. Give each player exactly one persistent, quota-limited SSH box whose runtime is fully restartable without manual cleanup.
4. Integrate AWS-compatible services (S3, DynamoDB, SQS) through small adapters, demonstrated locally against Floci with dummy credentials.
5. Ship a Lua 5.4 SDK covering observations, movement, targeting, firing, equipment, scans, dodging, and reconnect handling.
6. Field credible server-bot opponents: four difficulties and four personalities, including a mixed mode that assigns each bot a stable persona derived from its robot ID.
7. Provide a public surface that needs no sign-in: live spectating, match history, replays, profiles, and a duel leaderboard.
8. Keep deployment realistic for one VPS: Docker Compose, environment-driven configuration, no Kubernetes or multi-region machinery in Phase 1.

Literature review

Work / platform	Year	Idea drawn on	How this project differs
Robocode (IBM alphaWorks)	2001	Programming robots in a familiar language and battling them.	Robocode is a desktop Java game. We host the runtime in the cloud, persist scripts as cloud objects, and stream matches to anonymous viewers.
MIT Battlecode	2006–	Team strategy via code, tournament ladder with replays.	Battlecode centers on a yearly tournament toolchain. We target continuous play: a public queue, ratings, and always-on spectating on one VPS.
Two Sigma Halite	2016–2018	Leaderboard-driven AI competition with a server-side engine.	Halite ran batch submissions. Our robots are long-lived connected agents reacting tick-by-tick over WebSockets.
CodinGame	2012–	Browser-based multi-player programming battles.	CodinGame owns the runtime. Here players own a real SSH workspace and deploy from it, mirroring cloud development workflows.
Gambetta, Fast-Paced Multiplayer	2014	Client interpolation, server authority, reconciliation.	Applied at 10 Hz: browsers interpolate the last two snapshots; agents never hold state the server does not recompute.
Bernier, GDC	2001	Latency compensation in client/server game protocols.	Informs the 30 s agent reconnect grace and last-intent reuse during brief disconnects.
LocalStack	2017–	Faithful local AWS emulation for development.	Same philosophy through Floci locally, with adapter code written so the same calls hit real AWS later.
Google Agones	2017–	Game-server lifecycle on Kubernetes.	Deliberately not adopted: Phase 1 needs one VPS and Docker Compose, not a fleet control plane.

Gap addressed: no existing platform combines persistent per-player SSH development boxes, an authoritative deterministic watchable simulation, and AWS-compatible persistence in a package small enough for a course project on student hardware. That combination is the contribution of this prototype.

Architectural diagram



Data-flow notes.

- Browsers are dumb mirrors: they render snapshots and never compute state.
- Each box maintains exactly one outbound authenticated WebSocket; identity is claimed by the box-supervisor, not by browser-supplied credentials.
- The engine filters each agent's view to its robot's vision range (320 units by default; scope ×2, scanner ×1.75 with cloak reveal) while browser snapshots intentionally stay full-visibility.
- Every match carries a seed; procedural maps, item rolls, criticals, and bot randomness all derive from it, so replays are byte-stable.

Modules

Module	Location	Responsibility
Web client	apps/web	Svelte 5 arena canvas, telemetry, enrollment, box console, portal (play, matches, spectate, profile, leaderboard, docs).
API server	cmd/server, internal/api	REST + WebSocket hub, match lifecycle, duel queue, agent protocol, snapshot fan-out, admin status.
Simulation engine	internal/engine	Maps (starter + seeded procedural), obstacles, items, weapons, mines, turrets, hazards, three-stage zone collapse, power surges, over-time, events.
Bot AI	internal/engine/bot.go	Deterministic state machine (HUNT / COMBAT / HIDE / ESCAPE / ITEM / PATROL), difficulties dummy→sharpshooter, personalities aggressive/evasive/camper/mixed, wander bursts against stuck loops.
Lua runtime + SDK	internal/robotlua, sdk/lua	Player-facing Lua 5.4 SDK: observations, movement, aiming helpers, scans, dodging, zone status, reconnects; agent-side controller protocol.

Module	Location	Responsibility
Box provisioning	internal/boxes, cmd/provisioner	Per-player Docker SSH boxes, workspace quotas, agent startup, release flows.
Box supervisor	cmd/box-supervisor	In-box daemon: holds robot credentials outside the workspace, runs the saved start command, restarts on config change.
Cloud adapters	internal/cloud	S3 script snapshots, DynamoDB match persistence, SQS match jobs — one small interface per service so Floci can become real AWS by config.
Auth	internal/auth	WorkOS AuthKit sessions; public reads anonymous, mutations gated.
Data model	internal/model	Match, robot submissions and summaries, player stats, ratings, replays.
Script templates	internal/scripts, examples/	Six curated persona scripts deployable from the box console; embedded via go:embed, mirrored byte-identically in examples/.

Status of work

Review 1 core demo (complete)

- ✓ Two browser sessions receive synchronized 10 Hz simulation state.
- ✓ Two persistent boxes provisioned; each `/workspace/main.lua` edited over SSH; both robots registered into the same arena.
- ✓ Lua snapshots stored to S3, match jobs queued through SQS, box metadata and results in DynamoDB via Floci.
- ✓ Robots move and attack according to their scripts; agents are disconnected from identity and cannot report their own state.
- ✓ Stack stops and restarts without manual cleanup of source files or state.

Expanded demo (complete)

- ✓ Practice match against a server bot; obstacle and pickup rendering.
- ✓ Same match opened logged-out (public spectating) and inspected as a recap after completion.

Platform features (complete)

- ✓ Eight enlarged starter maps plus three seeded procedural styles (maze, rooms, bunkers) with connectivity checks.
- ✓ Typed item spawn zones, weapons, shields, dash cells, cloaks, mines, neutral turrets, hazards.
- ✓ Three-stage zone collapse, overtime, power surges, kill-streak rewards, drop-on-death loot.
- ✓ Duel queue (bot fill after 45 s, connection grace, automatic requeue), squad 5v5, solo free-for-all.
- ✓ Replays, player statistics, duel ratings, leaderboard, profiles, match history, admin status.
- ✓ Withdraw flow, box release flow, 30 s agent reconnect with last-intent reuse, Lua syntax validation on deploy.

Next review status

Planned item	Priority	Notes
Obstacle-aware bot movement	P1	Grid A* or potential fields so bots route around cover instead of probing.
Match-creatable arena size	P0	Small/medium/large/custom picker with clamps; engine constants become per-match config.
Follow cam + auto-director	P1	Viewer-locked camera, auto-switching on kills, pan/zoom for large arenas.
Share preview cards	P2	Match links render a title card even when opened logged out.
Supply airdrop event	P2	Timed high-tier item drop announced to every robot.
Equipment loadout slots	P2	Pre-match primary weapon, utility, and passive configuration.
Local headless simulator	P2	Run the engine against a script without the server for fast iteration.
Firecracker microVM isolation	Later	Requires Linux/KVM and adversarial testing; replaces trusted-reviewer Docker boxes.

Conclusion

Phase 1 demonstrates a complete cloud-native programmable-robot platform on local AWS-compatible infrastructure: persistent per-player development boxes, an authoritative deterministic simulation, cloud persistence for scripts and results, and a public viewing surface with replays and ratings. The architecture deliberately keeps every cloud dependency behind a small adapter, so the same code moves from Floci to real AWS and from Docker boxes to Firecracker microVMs as later phases demand. The next review focuses on navigation intelligence, viewer experience, and match-creation flexibility.

References

1. M. Nelson, “Robocode,” IBM alphaWorks, 2001. robocode.sourceforge.io
2. MIT Battlecode, annual programming competition. battlecode.org
3. Two Sigma, “Halite — an AI programming challenge,” 2016–2018. halite.io
4. CodinGame, multiplayer programming platform. codinggame.com
5. G. Gambetta, “Fast-Paced Multiplayer (client-server game architecture),” 2014. gabrielgambetta.com
6. Y. Bernier, “Latency Compensating Methods in Client/Server In-game Protocol Design and Optimization,” Game Developers Conference, 2001.
7. LocalStack, local AWS cloud stack emulator. localstack.cloud
8. Google, “Agones — open-source game server hosting on Kubernetes.” agones.dev
9. R. Ierusalimschy, L. H. de Figueiredo, W. Celes, “Lua 5.4 Reference Manual,” PUC-Rio, 2020. lua.org/manual/5.4
10. The Go Programming Language documentation. go.dev/doc
11. Svelte 5 documentation (runes and reactive primitives). svelte.dev/docs
12. Docker documentation. docs.docker.com